

תהליכונים

מערכות הפעלה
לינוקס

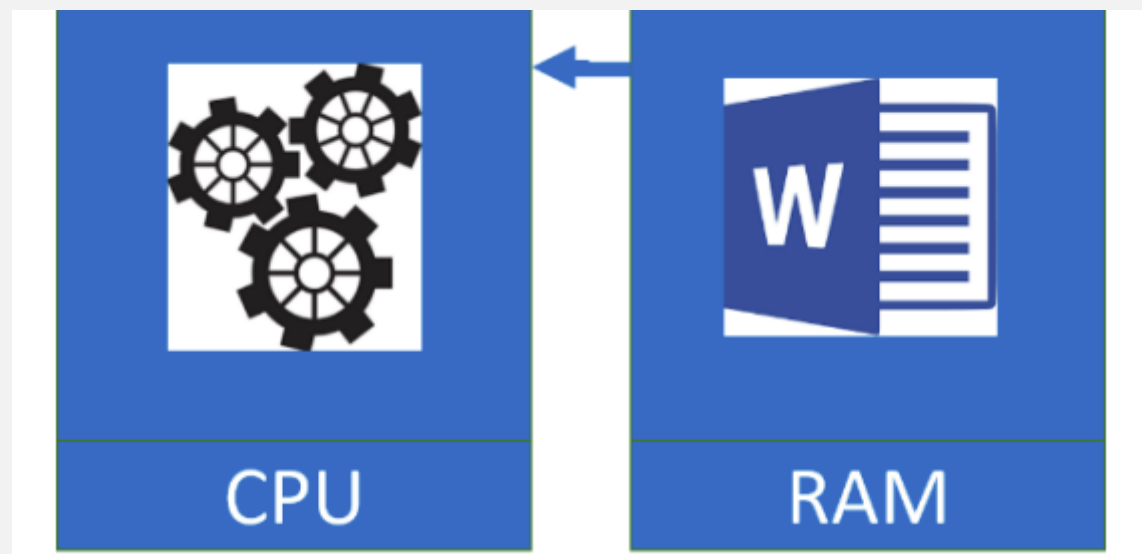
תהליכונים

בפרקים הקודמים ראינו כי תוכנות שרצות במחשב שלנו נמצאות תחת Process. Process הם חלק מרכזי בהבנת אופן הפעולה של מערכות הפעלה. בחלק זה נבין לעומק מהם Process, מהם Thread ונלמד לכתוב תוכנית ב-C שיוצרת Process וThread. נתחיל ממושגי היסוד – Process וThread.

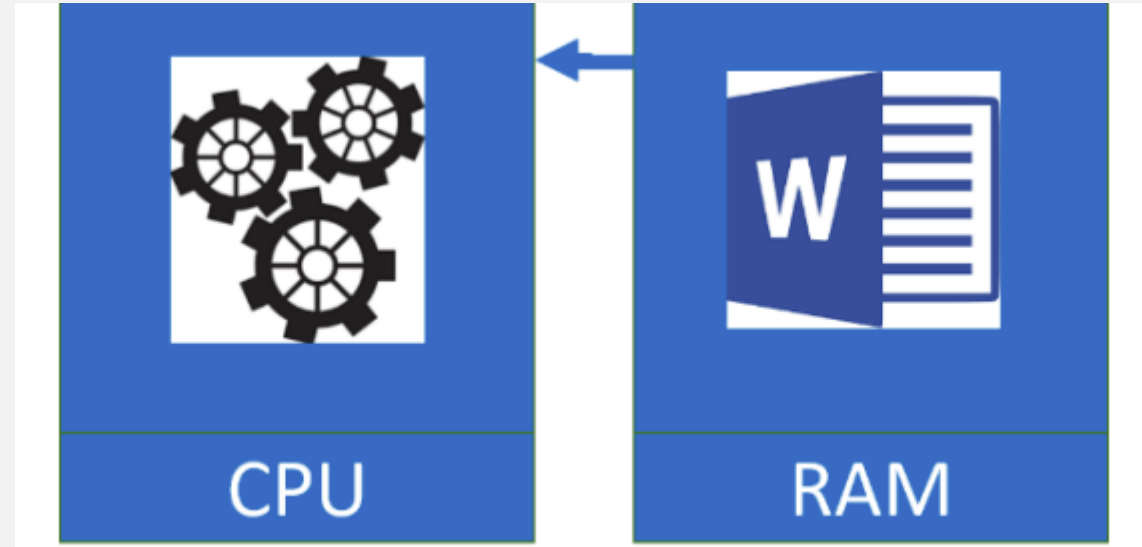
תהליכונים

תהליכונים - Threads

המושג Thread, או בעברית "תהליכון", מייצג רצף של פקודות שמבצעות משימה אחת. לדוגמה, ספר נכתב באמצעות תוכנת Word. אוסף כל הפקודות ש-Word מריץ הוא Thread של התוכנה Word. פקודות המכונה המרכיבות את התוכנה Word שמורות בזיכרון ה-RAM ומשם הן מועברות למעבד, שמריץ אותן.



תהליכונים



זהו המודל הפשוט ביותר של עבודת המחשב. מעבד שמריץ פקודות מהזיכרון. לו היינו מוסיפים לאיור יחידת קלט / פלט, היינו מקבלים את מודל המחשב של פון נוימן. אולם, למודל עבודה זה יש בעיות והוא אינו יעיל.

תהליכונים

ריבוי תהליכונים - Multi Threads

מתוך חבילת Sysinternals, הפעילו את התוכנה "Process Explorer". יחד איתה הפעילו את תוכנת Word. תוכנת Process Explorer מציגה לכם סטטוס עדכני של כל ה-Processes שרצים על המחשב. נחפש את ה-Process של Word. כדי לעשות זאת, נשתמש באייקון של המטרה שנמצא במסך ה-Process Explorer. המטרה היא האייקון ביותר בשורה:



תהליכונים

לחצו עליה לחיצה ארוכה עם העכבר ואל תשחררו. האייקון של העכבר הופך למטרה. גררו את המטרה אל מסמך ה-Word שפתוח אצלכם ושחררו את העכבר. ברגע שתעשו זאת, Process Explorer מיד יציג לכם את השורה המבוקשת, ה-Process של Word. כפי שאתם שמים לב, שם ה-Process הוא WINWORD.EXE.

 POWERPNT.EXE	< 0.01	175,856 K	159,596 K	35152 Microsoft PowerPoint	Microsoft Corporation
 WINWORD.EXE	0.01	205,016 K	247,836 K	40684 Microsoft Word	Microsoft Corporation
 SnippingTool.exe	0.03	5,764 K	29,384 K	29396 Snipping Tool	Microsoft Corporation

הקליקו על WINWORD הקלקה כפולה, וכעת עברו לטאב שנקרא Threads.

WINWORD.EXE:40684 Properties

Image Performance Performance Graph GPU Graph Threads TCP/IP Security Environment Strings

Count: 30

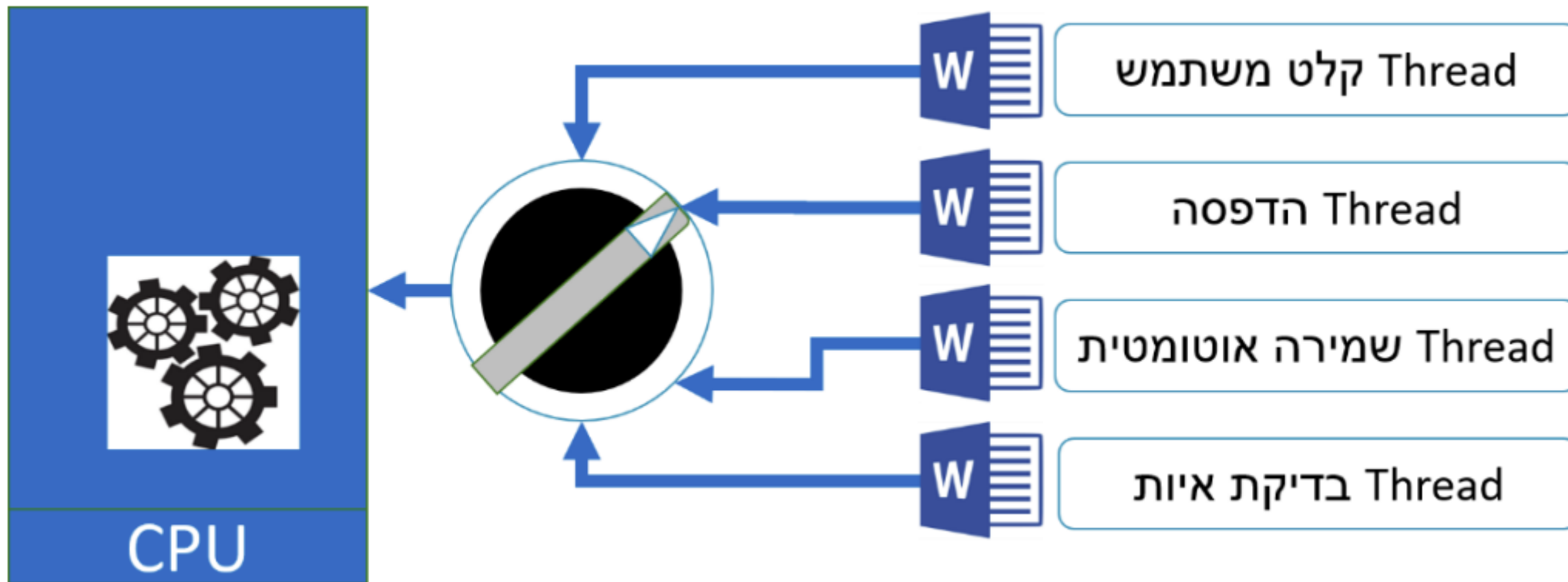
TID	CPU	Cycles Delta	Start Address
39776	< 0.01	79,586	mso20win32client.dll!Ordinal592+0xc0f
41444	< 0.01	67,817	mso20win32client.dll!Ordinal592+0x39e
38960	< 0.01	13,086	mso20win32client.dll!Ordinal592+0xc0f
41928			ntdll.dll!TplsTimerSet+0x40
19564			WINWORD.EXE+0x1000
40588			mso40uiwin32client.dll!Ordinal2987+0x3798
43076			mso40uiwin32client.dll!Ordinal2022+0x18d
43192			ntdll.dll!TplsTimerSet+0x40
38584			directmanipulation.dll+0x1de50
20020			Windows.UI.Input.Inking.dll!CreateDCompositionDirectInkFactoryPartner+0x8...
40804			combase.dll!CLSIDFromProgID+0x560
44544			Windows.UI.Input.Inking.dll!CreateDCompositionDirectInkFactoryPartner+0x7...
42924			ucrtbase.dll!_o____lc_collate_cp_func+0x10
42172			ucrtbase.dll!_o____lc_collate_cp_func+0x10
37400			ntdll.dll!TplsTimerSet+0x40
39708			ntdll.dll!TplsTimerSet+0x40
40496			mso20win32client.dll!Ordinal592+0xc0f

תהליכונים

הפתעה. מסתבר שה-Word הצנוע שלנו משתמש בלא פחות מאשר 30 Thread. מה הסיבה לכך? ובכן, לכל תוכנה, וגם ל-Word, יש "רגעים מתים". רגעים בהם התוכנה לא מבצעת דבר אלא ממתינה למשהו שצריך להתרחש. לדוגמה, אחת לכמה זמן תוכנת Word מבצעת שמירה אוטומטית של המסמך בדיסק הקשיח, ולכן עליה להמתין לסיום השמירה בדיסק. לדוגמה, התוכנה ממתינה להקלדות המשתמש על המקלדת. יחסית לקצב העבודה של המעבד, למשתמש לוקח זמן עצום בין הקלדה להקלדה.

אילו Word היה מריץ רק Thread אחד, אי אפשר היה לעשות הרבה כדי לשפר את היעילות. אולם, למה שלא נריץ מספר Thread של Word? יהיה לנו Thread שקולט את הקלדות המשתמש. Thread נוסף יבצע שמירה אוטומטית אחת לזמן מה. Thread שלישי יבצע בדיקת שגיאות הקלדה ו-Thread רביעי יבצע משימות של הדפסה ברקע, כך שנוכל להמשיך להקליד בעוד המסמך שלנו מודפס. הרבה יותר יעיל. כמובן שהחלוקה הזו ל-Thread היא רק דוגמה עבור Word, יש תוכנות רבות שיכולות להרוויח מחלוקת משימות ל-Thread.

תהליכונים



תהליכונים

איך נאפשר עבודה של מספר Thread'ים במקביל? הרי כל Thread מעביר למעבד פקודות מכונה שונות. נשים ביניהם לבין המעבד רכיב תזמון, נקרא לו "Scheduler". רכיב התזמון יעניק את משאבי העיבוד כל פעם ל-Thread אחר. בקרוב נראה שה-Thread האחר לא חייב להיות אפילו שייך לאותה התוכנית. כך, כאשר ה-Thread שמקבל קלט מהמשתמש יהיה חסר עבודה, יוענקו משאבי המעבד ל-Thread אחר שזקוק להם. באופן זה ניתן יהיה לבצע את כלל המשימות בצורה יעילה ומהירה יותר. הידד!

אלא שכעת נוצרה לנו בעיה חדשה: לכל הThread'ים שלנו יש דברים משותפים. מה משותף לכולם? הם ניגשים לאותו קובץ, לכן אם Thread מסויים לא חולק את הקובץ יתר הThread'ים לא יכולים לבצע את העבודה שלהם. הם ניגשים לאותו חלון של המשתמש, כך שאם המשתמש מקליד על תו מסוים או מבצע פעולה כלשהי, כל הThread'ים צריכים לדעת כך. בנוסף המסמך בגרסתה העדכנית צריך להיות בזיכרון של כל הThread'ים, כי אחרת הם לא יעבדו על אותה גרסה של המסמך.

תהליכונים

אילו משאבים צריכים להיות משותפים לכלל Threadים של אותו Process?

כתובות בזיכרון: כל Threadים רואים את אותם ערכים באותן כתובות בזיכרון. לדוגמה, אם Thread א' רואה בכתובת 0x402000 את הערך 5, אז Thread ב' יראה את הערך 5 באותה הכתובת.

זיכרון Heap: לכל Threadים יש זיכרון Heap יחיד. כלומר, כאשר Thread כלשהו מבצע הקצאת זיכרון malloc, כל Threadים מקבלים גישה לזיכרון הזה.

קוד הרצה: כל Threadים רצים מאותו אזור זיכרון. כיוון שמדובר באותה תוכנה, אין צורך להעתיק את אותו קוד שוב ושוב לזיכרון עבור כל Thread בנפרד. הקוד נמצא במקום יחיד וכל Thread מריץ את שורות הקוד שהוא זקוק להן.

משתנים סטטיים וגלובליים: כפי שראינו משתנים סטטיים וגלובליים נשמרים באותו אזור בזיכרון, ה-data section. כיוון שThreadים חולקים כתובות בזיכרון, הם גם חולקים ביניהם את כל המשתנים הסטטיים והגלובליים.

תהליכונים

אילו משאבים צריכים להיות ייחודיים לכל Thread?

רגיסטרים: ה-Thread אינם חולקים ביניהם את הרגיסטרים. במילים אחרות, כאשר Thread ב' מחליף את Thread א', הערכים שנמצאים בכל הרגיסטרים צריכים להתחלף.
מכך שכל הרגיסטרים משתנים, אפשר להבין שבאופן ספציפי משתנים גם הרגיסטרים EIP ו-ESP (נשתמש בשמות המוכרים עבור רגיסטרים של 32 ביט, הכוונה זהה עבור מערכות של 64 ביט עם הרגיסטרים RIP ו-RSP).

מצביע הקוד: כזכור הרגיסטר EIP מצביע על הפקודה הבאה שצריכה להיות מורצת על ידי המעבד. הגיוני שהרגיסטר EIP יתחלף בין Threadים, כי הרי כל Thread מריץ חלק אחר של הקוד.

תהליכונים

אילו משאבים צריכים להיות ייחודיים לכל Thread?

המחסנית: הרגיסטר ESP מצביע על מיקום ה-Stack, המחסנית של ה-Thread. הגיוני שלכל Thread תהיה מחסנית משלו, מכיוון שהמחסנית כוללת את שרשרת הקריאות לכל הפונקציות, וכיוון שכל Thread מריץ קוד שונה יש לו שרשרת קריאות שונה על המחסנית.

למעשה, לכל Thread יש מחסנית נוספת, שמשרתת את ה-Kernel. ההפרדה בין מחסנית של ה-Thread למחסנית של ה-Kernel נחוצה לטובת שמירה על מעגלי האבטחה. אילו היתה רק מחסנית אחת שמשרתת את שניהם, היה אפשר להשתמש בטכניקות שונות כדי להריץ קוד ברמת הרשאה של Kernel. לדוגמה, לערוך את הערכים שנמצאים על המחסנית כך שפונקציות של ה-Kernel יקבלו ערכים שונים מכפי שהן היו אמורות לקבל, או לשתול כתובות חזרה כך ששרשרת הקריאות לפונקציות של ה-Kernel תריץ בשלב מסוים קוד של ה-Thread, וכך קוד שאמור להיות מורץ ב-Userland יורץ בהרשאות Kernel.

תהליכונים

אילו משאבים צריכים להיות ייחודיים לכל Thread?

משתנים לוקליים: כתוצאה מכך שכל Thread הוא בעל מחסנית אחרת, ניתן להסיק שבניגוד למשתנים גלובליים וסטטיים, משתנים לוקליים – שגם הם נשמרים על המחסנית - אינם משותפים ל-Thread'ים. כלומר, אם Thread כולל לדוגמה פונקציה שבתוכה מוגדר משתנה מקומי i המשמש בתור אינדקס ללולאת for, אף Thread אחר אינו מכיר את המשתנה i .

אפשר לתהות- הרי אמרנו שהזיכרון הוא משותף לכל ה-Thread'ים, והמחסנית היא בסך הכל אזור בזיכרון, אם כך איך יכול להיות שהמחסנית אינה משותפת לכל ה-Thread'ים? אמנם המחסניות של כל ה-Thread'ים נמצאות באותו מרחב זיכרון, ולכן תיאורטית Thread יכול לשנות את המחסנית של אחר, אך באופן מעשי כדי לשנות ערכים במחסנית צריך לדעת היכן נמצאת המחסנית. מצביע המחסנית, ESP, משתנה בין Thread'ים ולכן לא פשוט ל-Thread לגשת למחסנית שאינה שלו.

תהליכונים

אילו משאבים צריכים להיות ייחודיים לכל Thread?

אזור זיכרון Thread Local Storage (בקיזור TLS): דמיינו שיש לנו Thread כלשהו, שמבצע חישוב מסויים באמצעות משתנה גלובלי, או סטטי. המשתנה הזה נמצא באזור הזיכרון של כל ה-Thread ששייכים לאותו Process ויש סכנה ש-Thread כלשהו ישנה את ערכו של המשתנה. פתרון אפשרי הוא להגדיר אותו כמשתנה לוקלי, על המחסנית, אבל לפעמים מאד נוח לנו להגדיר משתנה גלובלי. לדוגמה, הדבר מאפשר לכל הפונקציות שבקוד לגשת אליו. קיימת אפשרות נוספת, והיא להגדיר את המשתנה בתוך TLS, וכך למנוע מ-Thread אחרים לגשת אליו.

תהליכונים

אילו משאבים צריכים להיות ייחודיים לכל Thread?

עדיפות: לכל Thread יש עדיפות כלשהי, שה-Scheduler משתמש בה כדי להקצות זמן עיבוד. ב-Windows העדיפות היא מספר בין 0-31, כאשר מספר גבוה יותר משמעותו עדיפות גבוהה יותר. העדיפויות מתחלקות לשני תחומים: התחום בין 0-15 מיועד לעדיפויות רגילות, התחום בין 16-31 מיועד ל-Thread'ים שצריכים לרוץ בזמן אמת. המשמעות של ריצה בזמן אמת היא שחייב להיות מובטח שה-Thread יקבל זמן מעבד כל פרק זמן מוגדר, אחרת עלול להיות כשל. לדוגמה, נחשוב על שרת אינטרנט. השרת מחובר לכרטיס רשת, שמעביר לו מידע שהשרת צריך לפענח ולהשיב לו. המידע שמגיע מכרטיס הרשת מועבר לאזור זיכרון בגודל מסויים. אם השרת לא ימשוך את המידע מכרטיס הרשת כל פרק זמן מוגדר, אזור הזיכרון יתמלא, המידע החדש שמגיע יילך לאיבוד והשרת לא יוכל לטפל בו, לא משנה כמה משאבי מעבד יוקצו לעיבוד המידע. כלומר בעוד Thread'ים שרצים באופן רגיל צריכים לקבל *בממוצע* זמן מעבד מספק, Thread'ים שרצים בזמן אמת צריכים לקבל זמן מעבד על פרק זמן מוגדר.

בשורה התחתונה, אנחנו צריכים שחלק מהמשאבים של Thread יהיו משותפים. אם הם לא יוכלו לשתף המשאבים ביניהם, במקום לקבל ריצה חלקה של Thread שחולקים ביניהם את העבודה, נקבל אוסף Thread שמתנגשים זה בזה, מחכים זה לזה ולא עובדים על אותה גרסה של המסמך.

משאבים שאינם משותפים בין Thread של אותו Process	משאבים משותפים בין Thread של אותו Process
ערכי רגיסטרים. ספציפית מצביע הקוד ומצביע המחסנית	זיכרון (יתר המשאבים בטבלה נגזרים מכך שהזיכרון הוא משותף)
Thread Local Storage	זיכרון Heap
Thread ID	קוד הרצה
עדיפות	משתנים גלובליים וסטטיים

כאשר אנחנו מבינים ש- Multi Threads הוא רעיון נפלא אך זקוק למנגנון שידאג לכך שחלק המשאבים יהיו משותפים, זה הזמן לדון בProcessים.

תהליכונים

תהליכים - Processes

ה-Process מכיל את כל המשאבים המשותפים לכלל ה-Threads של אותה התוכנה. לדוגמה, כל הזיכרון ששייך לכל ה-Thread של Word שפתוחים על אותו מסמך, יהיה תחת Process אחד. שימו לב לכך, שאם נפתח מסמך Word נוסף יהיה לו Process אחד עם Thread אחרים.

אפשר לדמות את ה-Process למיכל של Thread. המיכל מכיל את כל מה שצריכים Thread בשביל לרוץ - כל המשאבים המשותפים יהיו מרוכזים אצלו.

כיוון ש-Process הוא רק מיכל, אין לו אפשרות להריץ קוד בעצמו. כל Process חייב Thread אחד לפחות שיריצ את הקוד. במילים אחרות, כאשר אנחנו מקליקים על תוכנה כדי להריץ אותה, נפתח Process חדש שמכיל לפחות Thread אחד. זהו ה-Main Thread, ממנו ניתן ליצור Thread נוספים.

תהליכונים

כל Process חדש שנפתח מקבל אוסף משאבים, אותם הוא חולק עם כל ה-Threads שיש לו. אך מהם אותם משאבים?

קוד הרצה שמועתק ל-RAM: כאשר אנחנו מפעילים Process מסויים, כל ה-Threads שהוא מפעיל חולקים את אותו הקוד. כל Thread יכול להשתמש בחלק אחר של הקוד, לדוגמה עבור Word יהיה Thread שמשתמש בקטע הקוד שמבצע בדיקת איות ו-Thread אחר ישתמש בקטע הקוד שמבצע שמירה של המסמך לדיסק הקשיח.

Thread אחד לפחות: כאמור Process הוא רק מיכל, אוסף של משאבים, ואינו יכול להריץ בעצמו קוד. על כן כל Process נפתח עם Thread אחד לפחות. כל ה-Threads של Process הם משאבים של ה-Process. הקצאת זיכרון: כל Process מקבל כמות של זיכרון, שהוא חולק עם כל ה-Threads שנמצאים תחתיו.

תהליכונים

כל Process חדש שנפתח מקבל אוסף משאבים, אותם הוא חולק עם כל ה-Threads שיש לו. אך מהם אותם משאבים?

עדיפות: נושא העדיפות הוסבר כאשר דיברנו על עדיפויות של Thread-ים. ל-Process-ים שונים עשויה להיות עדיפות שונה, לדוגמה Process-ים שמבצעים משימות ניהול עבור מערכת ההפעלה צריכים לקבל עדיפות גבוהה יותר מאשר, לדוגמה, Process של כרום שמציג סרטון יוטיוב. כל ה-Thread-ים שנמצאים תחת אותו Process יורשים ממנו את רמת העדיפות הבסיסית, אך תוך כדי ריצה ניתן לשנות את העדיפות שלהם. לדוגמה ניתן להפוך Thread לרמת עדיפות של ריצה ברקע, או אפילו ל-IDLE (אינו מבצע דבר, העדיפות הנמוכה ביותר), אך גם ניתן להעלות את העדיפות של Thread-ים במידה מסויימת אפילו מעבר לעדיפות הבסיס של ה-Process.

תהליכונים

כל Process חדש שנפתח מקבל אוסף משאבים, אותם הוא חולק עם כל Thread שיש לו. אך מהם אותם משאבים?

מזהה ייחודי PID: לכל Process יש מספר מזהה ייחודי - Process ID, או בקיצור PID. אולי הבחנתם בכך כאשר עבדתם עם Process Explorer, ה-PID מוצג באחד הטורים.

טבלת Handle Table: נניח ש-Process רוצה לגשת למשאב מערכת או אובייקט של מערכת ההפעלה, כגון קובץ (בהמשך נכיר דוגמאות נוספות). אי אפשר לעשות זאת ישירות, אלא דרך אובייקט שנקרא Handle.

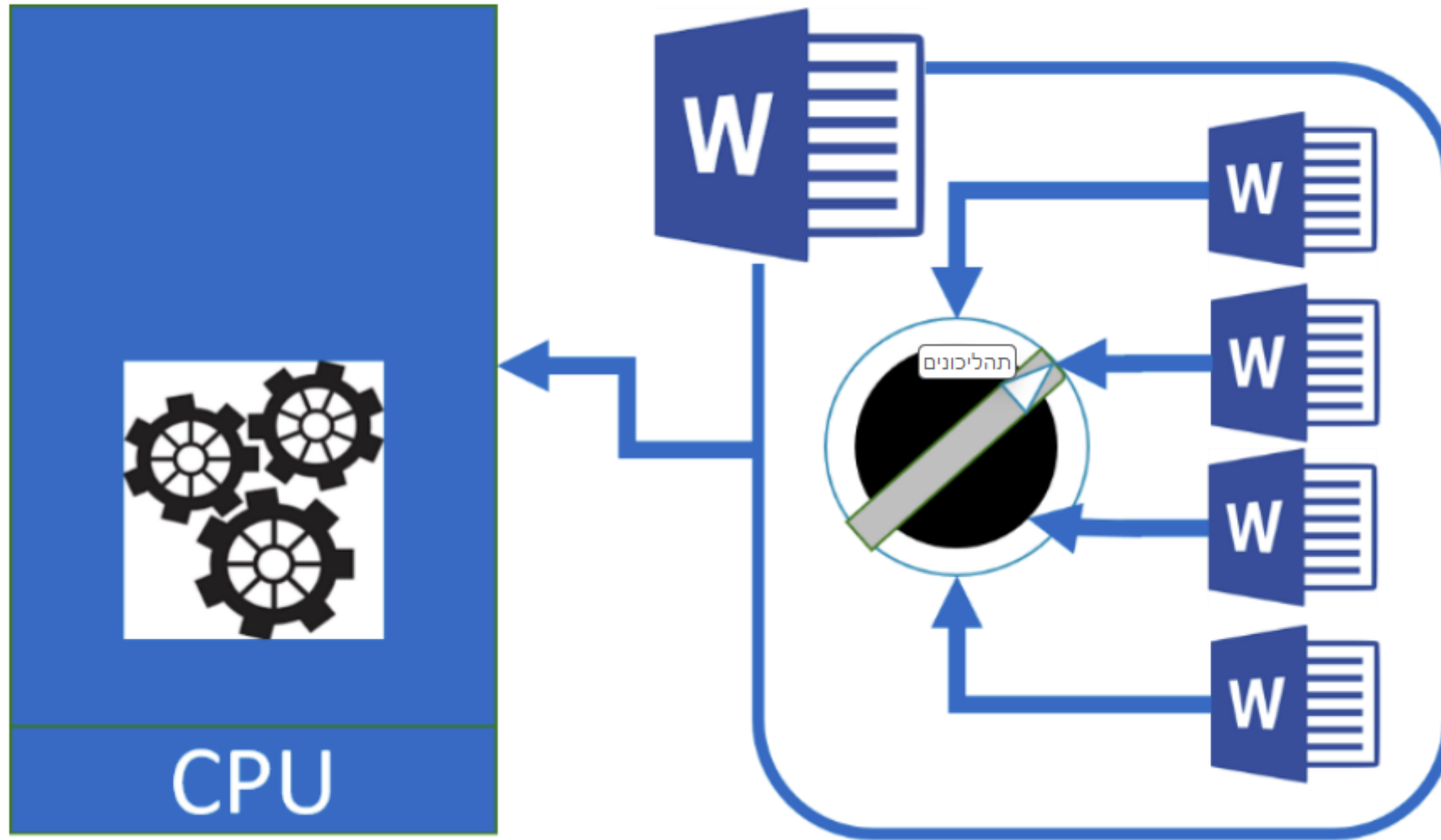
הרשאת אבטחה Security Token: מערכת ההפעלה שומרת רשימה של הרשאות האבטחה שנדרשות על מנת לגשת לכל משאב. לכל Process יש Security Token, שצריך להיות לפחות לפי רמת ההרשאה של משאב ברשימה כדי לגשת אליו.

תהליכונים

כעת נוכל לשדרג את ההסבר שלנו על אופן פעולת מערכת ההפעלה. עצרנו במודל של Multi-Threads, שבו מספר Thread ים צריכים לפעול במקביל תוך שיתוף משאבים זה עם זה. מושג ה-Process עונה על השאלה כיצד מתבצע שיתוף המשאבים.

כאשר אנחנו מריצים Process, הוא מקבל את כל המשאבים הנדרשים ממערכת ההפעלה. ה-Process פותח מספר Thread ים כפי שנדרש לו, וכולם חולקים את המשאבים. כל Thread מקבל משאבי CPU בתורו, שנקבע על ידי Scheduler של מערכת ההפעלה.

תהליכונים



תהליכונים

נסכם את ההבדלים בין Process ו-Thread:

Process	Thread
מקבל משאבים (זיכרון וכו') ממערכת ההפעלה	יורש משאבים מה-Process
לא משתף משאבים עם Process אחרים	משתף משאבים עם Thread אחרים תחת אותו Process
אין לו קוד שרץ, חייב לפתוח Thread	מריץ קוד
יכול להכיל כמה Thread ים	חייב שיהיה לו Process אחד שייצור אותו

תהליכונים

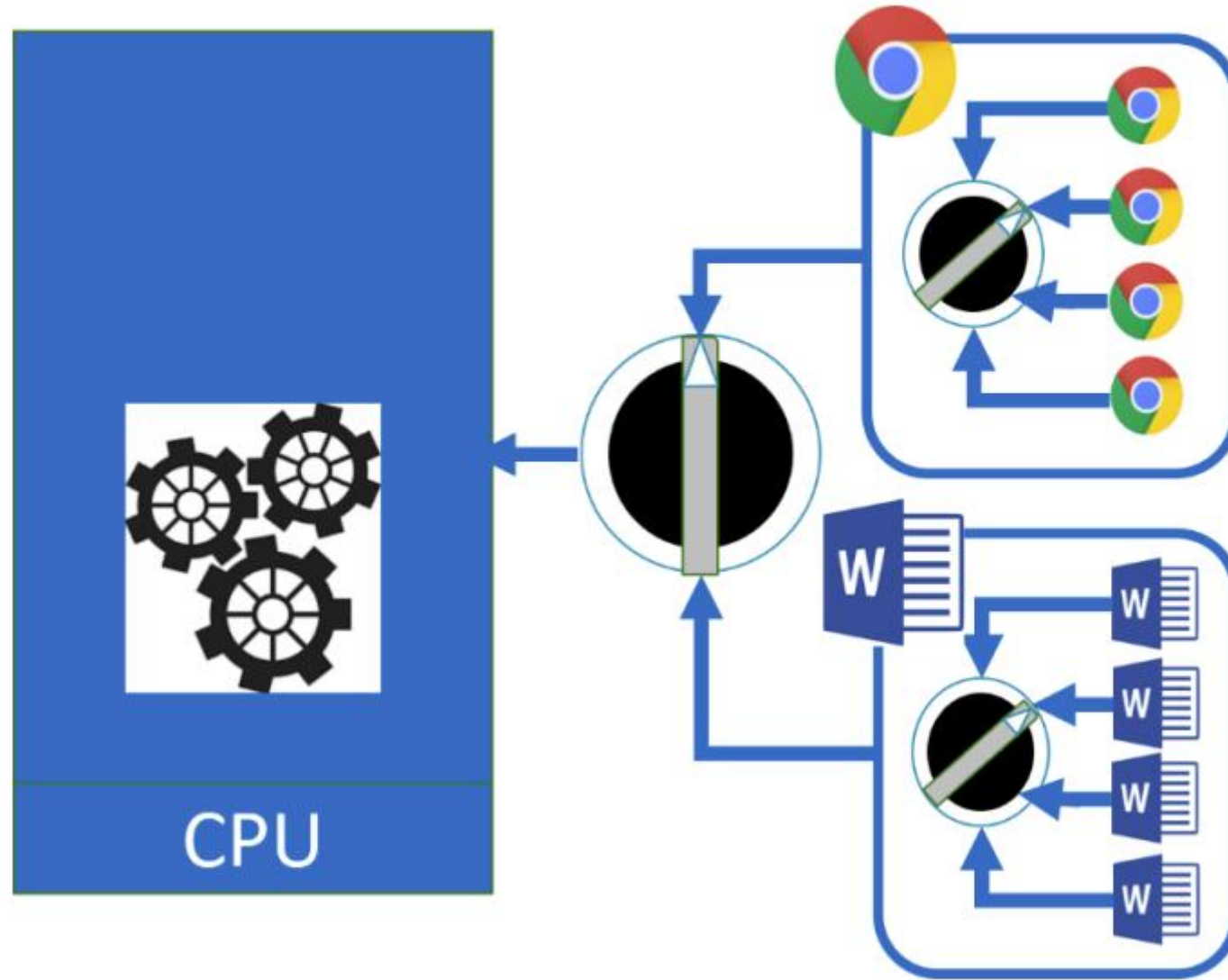
ריבוי תהליכים - Multi-Process

אנחנו מבינים ש-Thread ים ששייכים לאותו Process חולקים ביניהם את המשאבים של ה-Process, אולם Process יחיד לא פותר את כל הבעיות, כיוון שתוכנות רבות צריכות להיות מורצות בו זמנית.

הפתרון הוא כמובן עבודה עם Process ים רבים בו זמנית. כל Process יקבל את המשאבים הנחוצים לו וירוח באופן נפרד מה-Process האחרים.

האיור הבא ממחיש את אופן הפעולה של Process ים ו-Thread ים תחת מערכת ההפעלה. במערכת שבאיור קיימים שני Process ים: אחד עבור כרום ושני עבור Word. לכל Process יש מספר Thread ים. כדי לבחור מי מקבל את זמן המעבד, ה-Scheduler של מערכת ההפעלה בורר את ה-Process ובתוכו את ה-Thread.

תהליכונים



תהליכונים

נסכם בהבדל בין ריבוי Thread לריבוי Process: בריבוי Thread כל Thread ייגשים למרחב זיכרון בו קיים אותו קוד הרצה. ההבדל בין ה-Thread הוא בעיקר בנקודה בקוד שבה הם נמצאים. כמו כן מרבית המשאבים שלהם הם משותפים, הדבר מתבטא לדוגמה בכך ש-Thread יכול לשנות משתנה גלובלי שהוגדר על ידי Thread אחר.

בריבוי Process, לא לכל ה-Process יש בהכרח את אותו הקוד. כלומר, יכול להיות מצב ששני Process מריצים כרום, ואז יש להם את אותו קוד, אך זהו המצב החריג. בדרך כלל Process שונים מריצים קוד שונים. כמו כן לכל Process יש משאבים שונים, כאשר הכוונה בעיקר למרחב זיכרון שונה. אך האם הדבר אומר ש-Process רצים לחלוטין בנפרד זה מזה ואינם מסוגלים לשתף זיכרון? על שאלה זאת נענה בקרוב.

תהליכונים

ביצוע Context Switch

המשמעות של המונח Context Switch היא "כל הדברים שצריכים להתבצע על מנת לעבור ממשימה למשימה אחרת". לדוגמה, אם יש לנו בילוי עם חברים לאחר אימון ספורט, נצטרך לעשות Context Switch שיכלול מקלחת והחלפת בגדים. כמו שלוקח זמן להתקלח ולהחליף בגדים, לוקח גם זמן למערכת ההפעלה לבצע Context Switch בין משימות שהיא צריכה לבצע. בחלק זה נדון בדברים שמערכת ההפעלה צריכה לבצע כדי לעשות Context Switch בין Thread'ים.

תהליכונים

נניח שמערכת ההפעלה מבצעת Context Switch בין Thread'ים של אותו Process. מה הדברים שמערכת ההפעלה צריכה לשנות?

ראינו שעיקר ההבדל בין Thread'ים הוא הערכים שיש ברגיסטרים שלהם. בזמן שמערכת ההפעלה מבצעת החלפה בין Thread'ים של אותו Process, כל מה שהיא צריכה לעשות זה לשמור בצד את הערכים שיש ברגיסטרים ולטעון אל הרגיסטרים את הערכים המתאימים ל-Thread החדש. ברגע שהערכים החדשים טעונים, גם הערך של EIP משתנה כך שהוא מצביע על הפקודה הבאה שה-Thread צריך להריץ ושה-Thread עצר בה בפעם הקודמת שזמן המעבד ניתן לו. הערך של ESP גם הוא משתנה וכך ה-Thread החדש מקבל חזרה את הגישה למחסנית שלו, שנותרה ללא שינוי מאז הפעם הקודמת שה-Thread רץ. כלומר, ביצוע Context Switch בין Thread'ים של אותו Process הוא פעולה שדורשת זמן אך למרות זאת אינה מסובכת במיוחד.

תהליכונים

לעומת זאת, ביצוע Context Switch בין Thread ים שאינם באותו Process היא פעולה מורכבת הרבה יותר. הסיבה לכך היא ש-Thread ים שנמצאים ב-Process ים שונים אינם חולקים זיכרון זה עם זה. עדיין לא למדנו על זיכרון וכיצד הוא פועל, אבל לצורך הפשטות נניח שיש לנו זיכרון RAM בגודל 1MB ושתי תוכניות, שכאשר כל אחת מהן טעונה לזיכרון בנפרד היא תופסת 1MB, כלומר את כל זיכרון ה-RAM. לכן, על מנת לבצע Context Switch בין ה-Process ים שמריצים את שתי התוכניות, מערכת ההפעלה תצטרך למעשה להחליף את כל זיכרון ה-RAM של המעבד. בשלב הראשון מערכת ההפעלה תעתיק את כל זיכרון ה-RAM של Process א' אל הדיסק הקשיח. היא חייבת לעשות זאת מכיוון שאחרת יאבדו כל הערכים של המשתנים שנוצרו והשתנו בזמן הריצה. לאחר מכן להעתיק מתוך הדיסק הקשיח אל ה-RAM את הזיכרון של Process ב', כפי שנשמר בדיסק הקשיח בפעם האחרונה ש-Process ב' סיים את הריצה שלו.

תהליכונים

אפשר כמובן לטעון ששלב א', ההעתקה של כל הזיכרון של Process א' אל הדיסק הקשיח, יכול לעבור שיפור ביצועים ניכר. אפשר להעתיק חזרה לדיסק הקשיח רק את השינויים שה-Process ביצע תוך כדי ריצה. אך הדבר אינו משנה את השורה התחתונה, ככל שיש יותר העתקות זיכרון נדרש זמן רב יותר לביצוע Context Switch והמשתמש עלול להרגיש שהמחשב "איטי". זו גם הסיבה לכך שכדי שמחשב ירוץ במהירות, חשוב לא רק מהירות המעבד אלא גם גודל ה-RAM.

תהליכונים

שימוש ב-Multi-Process לעומת Multi-Thread

מתי תוכנה תשתמש ב-Multi-Processing ומתי ב-Multi-Threading?

נפתח Process Explorer. אם יש לכם דפדפן כרום שרץ עם כמה טאבים, תוכלו לראות תמונה שדומה לתמונה הבאה- תחת כרום יש Sub Processes שגם הם נושאים את השם כרום. כלומר, ה-Process כרום יצר Process'ים בנים באותו השם.

Process	CPU	Private Bytes	Working Set	PID	Description
 chrome.exe	0.01	416,744 K	354,032 K	36680	Google Chrome
 chrome.exe		1,796 K	2,348 K	29276	Google Chrome
 chrome.exe		1,988 K	2,328 K	35228	Google Chrome
 chrome.exe		766,112 K	281,088 K	37704	Google Chrome
 chrome.exe	0.05	46,668 K	50,056 K	30740	Google Chrome
 chrome.exe		47,848 K	15,056 K	19236	Google Chrome

תהליכונים

כמובן שלא רק כרום יוצר Sub Processes כאשר הוא פועל. דוגמה נוספת היא שירות המוזיקה Spotify:

Process	CPU	Private Bytes	Working Set	PID	Description
 Spotify.exe	0.90	140,100 K	93,776 K	11440	Spotify
 Spotify.exe	< 0.01	32,812 K	4,276 K	11576	Spotify
 Spotify.exe	0.18	120,932 K	66,496 K	11820	Spotify
 Spotify.exe	< 0.01	44,440 K	15,672 K	11944	Spotify
 Spotify.exe	0.57	174,108 K	125,728 K	12208	Spotify

ספוטיפיי פותח Process ים נוספים כדי לחלק את המשימות שעליו לבצע. לדוגמה, משימה אחת היא להציג למשתמש את הממשק הגרפי הכולל רשימת שירים ואפשרויות חיפוש. משימה זו דורשת עבודה מול הכרטיס הגרפי של המחשב. משימה אחרת כוללת עבודה מול כרטיס הקול והשמעה של המוזיקה. משימה אחרת היא תקשורת מול השרת של ספוטיפיי והזרמה של השירים אל המחשב.

תהליכונים

אך חלוקת משימות יכולה להתבצע גם על ידי Thread. למה ספוטיפי – ותוכנות רבות נוספות – מתעקשות לפתוח דווקא Process נוספים? למה לא לבצע הכל רק על ידי Multi-Threading?

ראשית, במקרים רבים חלוקת משימות אכן מתבצעת על ידי פתיחת Thread נוספים. הנה לדוגמה כרום, שלמרות שראינו שפותח Process נוספים, הוא משתמש גם בריבוי Thread. מתוך Process Explorer, הקליקו על ה-Process המבוקש ובתוכו הקליקו על הטאב Threads. תוכלו לראות את כל ה-Thread ש-Process עושה בהם שימוש:



chrome.exe:29276 Properties

Image Performance Performance Graph GPU Graph Threads TCP/IP Security Environment Strings

Count: 7

TID	CPU	Cycles Delta	Start Address
31904			chrome.exe!GetHandleVerifier+0xa6da0
26808			chrome.exe!GetHandleVerifier+0x5ae90
36524			chrome.exe!GetHandleVerifier+0x5ae90

תהליכונים

יש יתרונות רבים לחלוקת המשימות ל-Thread. כיוון שהם חולקים משאבים, קל יחסית לנהל משאבים משותפים. וכפי שראינו, ביצוע Context Switch בין Thread של אותו Process הוא משימה חסכונית יותר מאשר בין Process שונים.

אך גם לחלוקה ל-Process יש מספר יתרונות חשובים. היתרון הראשון הוא חוסר תלות- אם Process בן קורס עקב באג כלשהו, יתר ה-Processים ממשיכים לפעול. באופן ספציפי במקרה של כרום, הדפדפן מריץ תוספים שונים שמותקנים עליו. התוספים יכולים להיות חוסם פרסומות, תוספים שהאנטיוירוס התקין, או תוספים שפותחו לטובת שיפור חוויית המשתמש באתרים ספציפיים. התוספים הללו יכולים לבוא ממקורות שונים וגוגל מפתחת הכרום לא יכולה להבטיח שהתוספים הללו לא יקרסו תוך כדי ריצה. מבחינת כרום, הגיון לתת לכל תוסף לרוץ ב-Process משלו, כך שקריסה של תוסף לא תגרום לסגירת כל פעולת הגלישה שלנו. כמו כן, גם גלישה לדף אינטרנט עלולה "להתקע" או במקרים נדירים אפילו לקרוס. לכן גם לטאבים שונים שפתוחים בכרום מוקצים Processים שונים.

תהליכונים

היתרון השני של Multi Processing הוא בכך שרוב המחשבים הביתיים המודרניים כוללים מספר מעבדים. בדרך כלל ארבעה או שמונה. כאשר Process פותח מספר Thread, יותר יעיל להריץ אותם על אותו מעבד. מיד נבין מדוע. לכן חלוקה ל-Processים מאפשרת לנצל בצורה יותר טובה את ריבוי המעבדים שיש במחשב.

תהליכונים

אם כן, מדוע יש יתרון בהרצת Thread של אותו Process על אותו מעבד?

לכל מעבד יש זיכרון קטן

ומהיר שצמוד אליו. בניגוד ל-RAM שהוא משאב משותף לכל מעבדים, הזיכרון המהיר של כל מעבד שייך לו בלבד, נקרא לו באופן זמני "זיכרון פרטי". יש כל הזמן חילופי מידע בין ה-RAM המשותף לזיכרון הפרטי של כל מעבד. לב הבעיה בחלוקה של מספר Thread של אותו Process בין כמה מעבדים, היא בכך שאם כל מעבד משנה דברים בזיכרון הפרטי שלו, ולאחר מכן השינויים מועתקים אל ה-RAM המשותף, אז עלולה להיות התנגשות. לדוגמה בזכרון הפרטי של מעבד א כתוב שערכו של המשתנה הגלובלי X הוא 5, ואילו בזכרון הפרטי של מעבד ב כתוב שערכו של X הוא 4. מהו הערך ה"נכון" אותו יש לשמור ב-RAM? יש צורך ליצור מנגנון שימנע בעיות כאלה. קיים מנגנון כזה, אך פעולת התיאום לא מגיעה חינם אלא יש לה עלויות של זמן עיבוד. לכן, כאשר רוצים לחלק משימה באופן מיטבי בין מעבדים, יש יתרון לריבוי Process על פני ריבוי Thread.

תהליכונים

היתרון השלישי של ריבוי Processים הוא אבטחה. כיוון שהתחלנו את ההסבר שלנו מדפדפן כרום, נסקור בקצרה את מנגנון ה-Chrome Sandbox. אחת הבעיות שכרום נתקל בהן היא שאתרי אינטרנט עלולים להיות זדוניים ולנסות לגרום לדפדפן לבצע פעולות זדוניות. אם האקר מוצא באג בדפדפן הוא יכול לנצל אותה כדי לגרום לדפדפן להריץ קוד כרצונו. לדוגמה, להוריד תוכנה מסויימת לרשום אותה ב-Registry בלי לשאול את המשתמש. מוקדם יותר בפרק הזכרנו שלכל Process יש Security Token ושכאשר Process מבקש לגשת למשאב מסויים שדורש הרשאה, מערכת ההפעלה משווה את ה-Security Token לרמת ההרשאה של המשאב. עלול להיות מצב שהקוד הזדוני ינצל את ה-Security Token של כרום כדי לגשת למשאבים מאובטחים, כגון ה-Registry, כיוון שמבחינת מערכת ההפעלה מי שמבקש לגשת למשאב המאובטח הוא כרום. למערכת ההפעלה אין מושג שכרום מריץ קוד זדוני שדף אינטרנט גרם לו להריץ. הפתרון של כרום הוא לייצר Sandbox - "ארגז חול". זהו Process שיש לו את ההרשאות הנמוכות ביותר, כך שהוא חסר תועלת בשביל האקר בעל מטרות זדוניות.

תהליכונים

לסיכום, כפי שאנחנו רואים שמתרחש בפועל, תוכנות משלבות בין יתרונות של Multi-Thread ו-Multi-Processing על מנת לנצל בצורה המיטבית את משאבי העיבוד שנתונים לרשותם.

תהליכונים

Services / Daemons

המונחים Service ו Daemon הינם שמות נרדפים, כאשר Service מקובל בסביבת Windows ואילו Daemon בסביבת Linux. המונח מציין סוג מיוחד של Process שיש לו שלושה מאפיינים:

- א. מופעל אוטומטית על ידי מערכת ההפעלה
- ב. מנוהל על ידי מערכת ההפעלה
- ג. אין לו מסך משתמש, כלומר הוא רץ מאחרי הקלעים.

תהליכונים

בסביבת Windows הניהול של כל ה-Services מתבצע באמצעות Process שנקרא Service Control Manager, או בקיצור SCM. ה-SCM דואג בין היתר להפעיל אוטומטית את ה-Service, להחזיק מידע על הסטטוס של ה-Service ולהעביר אליו וממנו מידע.

נתעמק מעט בנושא ההפעלה האוטומטית. ברור שזהו הבדל משמעותי בין Service ל-Process "רגיל", כגון Word. כדי להפעיל Word צריך שהמשתמש יקליק על התוכנה של Word. לעומת זאת, Service רץ אוטומטית באמצעות ה-SCM. אם כך אפשר לשאול, מה ההבדל בין Service לבין Process שרץ בעזרת ה-Scheduled Tasks? נזכור, שכאשר למדנו על תוכנת Autoruns ראינו שאחת הדרכים לגרום לתוכנה לרוץ באופן אוטומטי היא להכניס אותה ל-Scheduled Tasks. אם כך, האם גם תוכנה שרצה אוטומטית באמצעות Scheduled Tasks היא בעצם Service?

תהליכונים

כאשר תוכנה רצה באמצעות Scheduled Task, אחת לזמן המוגדר לה היא "מתעוררת" ומבצעת את המשימה שלה, לאחר מכן נסגרת. יש בכך יתרון, מכיוון שבזמן שהתוכנה סגורה אין לה Process שדורש משאבים. זה מתאים למשימות מסוג מסויים. לדוגמה, לבדוק אחת ליום אם יש עדכון אנטיוירוס. לעומת זאת, Service רץ כל הזמן ויכול להגיב לאירועים שאי אפשר לדעת מראש מתי הם יקרו. נסקור מספר דוגמאות ל-Service ים, נסו לחשוב מה היה קורה אם התוכנות הללו היו "מתעוררות" רק אחת לזמן מה, ולא מסוגלות להגיב באופן מיידי לאירועים שמתרחשים, כגון משתמש שמבצע Log-In או חיבור של המחשב לרשת:

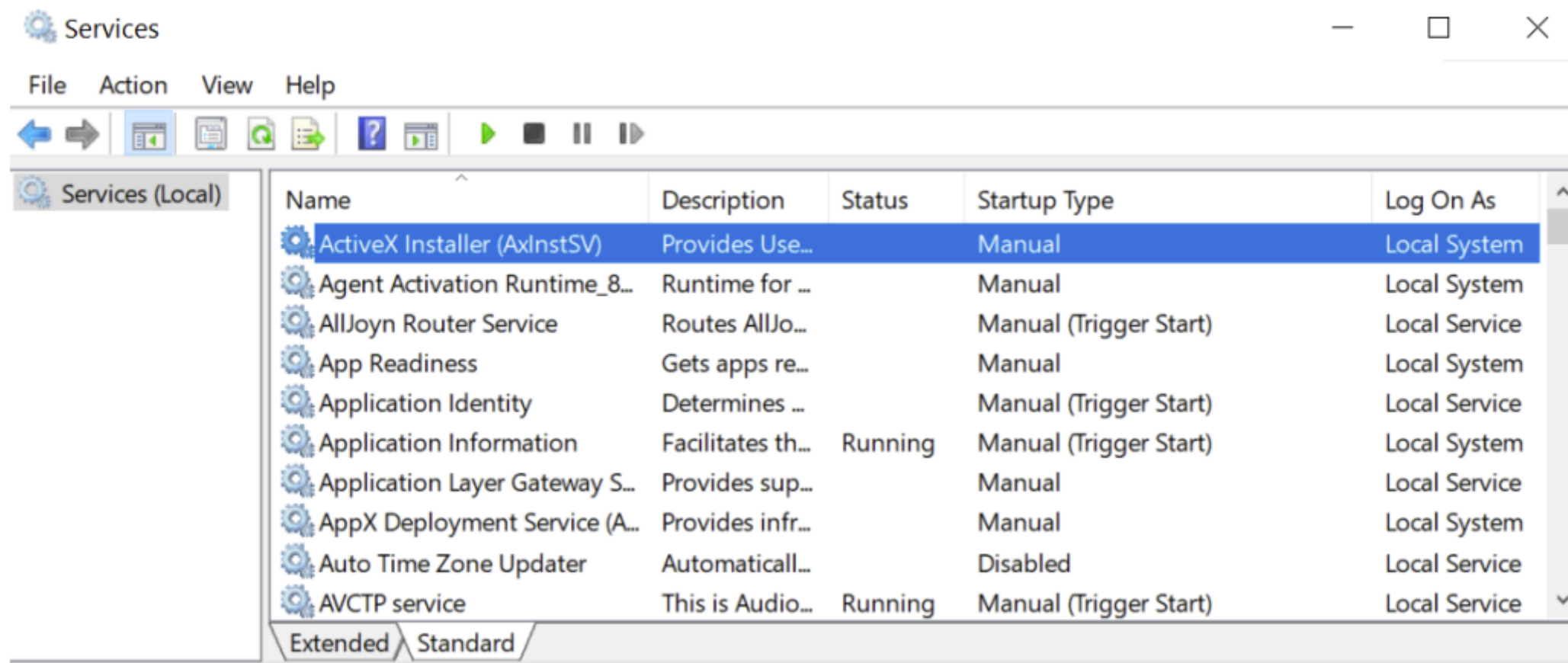
תהליכונים

דוגמאות ל-Service'ים:

- DHCP Client :פרוטוקול DHCP מאפשר למחשב לקבל כתובת IP, כתובת IP של שרת DNS, מסכת רשת וראוטר ברירת מחדל. המחשב נדרש לפעול מול שרת DHCP. על כן נחוץ שהמחשב יפעיל עם העליה שלו DHCP Client. אין טעם להטריח את המשתמש, לכן הפעולה מבוצעת מאחרי הקלעים כ-Service
- User Profile Service :כאשר אנחנו מבצעים Log In למחשב, באופן אוטומטי התצוגה עולה כפי שהיא שמורה בפרופיל שלנו ב-Registry, ויש שימוש בהרשאות ההרצה שלנו לתוכנות השונות שמותקנות על המחשב. הדברים מתבצעים על ידי Service, שהרי אין הגיון לבקש מהמשתמש להריץ תוכנה שתבדוק את ההרשאות שלו ותחסום גישה לתוכנות מסויימות...

תהליכונים

קיימים עוד Service ים רבים שרצים ממש כרגע על המחשב שלכם. מעוניינים לחקור אותם? כדי למצוא אותם, הקישו על WinKey+R ובחלון הרצת התוכנות שנפתח כיתבו `services.msc`.



תוכלו לבחור כל Service שרץ אצלכם ולקרוא עליו פרטים, כגון התפקיד שלו.